

REVIEW 2

Methodology · System Architecture

Smart Inventory & Warehouse Management System

Team Members

S Svara Haasini – 2520090170

B Sudharshini – 2520090227

Samyuktha Kandukuri – 2520090157

Database Systems Engineering & Distributed Backend Development (25CS1302E)

Under the Guidance of: Course Faculty Guide

Agenda

- Recap
- Proposed Methodology
- System Architecture
- Database Design (ER Model)
- Technologies Used

Recap — Review 1

- Literature survey: spreadsheets, barcode, POS, and ERP tools reviewed.
- Research gap: no low-cost, relational, multi-warehouse solution with automated alerts.
- Direction: a MySQL + Flask system with automated stock updates and reporting.

Proposed Methodology

- **Requirement Gathering**
- Identify entities: products, categories, suppliers, warehouses, stock, transactions.
- **Schema Design**
- Design normalized ER model with primary/foreign-key relationships.
- **Database Creation**
- Implement schema in MySQL using MySQL Workbench.
- **Backend Integration**
- Build Flask routes and Python connectors for CRUD operations.
- **Frontend & Testing**
- Connect Bootstrap UI; test stock-update and low-stock alert logic.

System Architecture — Layered View

- **Presentation Layer**
- HTML, CSS, JavaScript, and Bootstrap render forms and dashboards for products, suppliers, warehouses, and stock in the browser.
- **Application Layer**
- Flask handles requests, business logic, and validation, and routes data between the frontend and the database.
- **Data Access Layer**
- Python database connectors send SQL queries (insert, update, delete, select) to MySQL and return results to Flask.
- **Database Layer**
- MySQL stores structured tables for Products, Categories, Suppliers, Warehouses, Stock, and Transactions, linked through primary and foreign keys.

Database Design — ER Model

- **Product** (product_id, name, category_id, price, quantity) — linked to Category and Supplier.
- **Category** (category_id, category_name) — one category has many products.
- **Supplier** (supplier_id, name, contact_info) — supplies one or more products.
- **Warehouse** (warehouse_id, location, capacity) — holds stock for multiple products.
- **Stock** (stock_id, product_id, warehouse_id, quantity) — links Product and Warehouse (many-to-many).
- **Transaction** (transaction_id, product_id, type, quantity, date) — logs every stock movement for traceability.

Technologies Used

Category	Technology	Purpose
Database	MySQL, MySQL Workbench	Stores products, suppliers, warehouses, stock, transactions.
Backend	Python, Flask	Handles application logic and database connectivity.
Frontend	HTML, CSS, JavaScript, Bootstrap	Creates a clean, responsive user interface.
Tools	VS Code, Flask dev server	Development and local testing environment.